IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

TwinShield-Fusion: Digital-Twin Residual Gating for Detection of Unseen Attacks on Automotive CAN Buses

FIRST A. AUTHOR[1], SECOND B. AUTHOR[2], and THIRD C. AUTHOR[3]

[1]Department of Computer Science and Engineering, [Institution], [City], [Country]
[2]Department of Computer Science and Engineering, [Institution], [City], [Country]
[3]Department of Computer Science and Engineering, [Institution], [City], [Country]

Corresponding author: First A. Author (e-mail: author@institution.edu).

This work was carried out as part of an undergraduate research project. The GEM-CAN dataset is distributed under CC-BY-NC-3.0 and was obtained from Zenodo (10.5281/zenodo.17834776).

ABSTRACT An electric vehicle (EV) sends braking, steering and propulsion instructions to dozens of electronic control units (ECUs) that communicate via a broadcast bus called the Controller Area Network (CAN), which does not include a sender ID or authentication. Any node on the bus can send commands, and since the protocol cannot be changed without replacing all the ECUs, the only way to protect against intrusion is to use intrusion detection. Published detectors are evaluated against a fixed attack catalogue, and an attack not in the catalogue is evaluated against the least worst known attack class. We suggest the combination of a supervised classifier and a gated recurrent unit (GRU) digital twin trained on normal traffic, which we call TwinShield-Fusion. It is a standardised per-identifier, per-byte payload prediction residual thresholded at a calibrated false-positive rate. On GEM-CAN, 143,444 frames from a GEM e6 EV, with one attack class quarantined from each training stage, the baselines detect 0.84%, 0.00% and 25.21% of it, while TwinShield-Fusion detects 100.00% at 2.26% false positives. Repeating the protocol on SynCAN, a second and independent benchmark whose attacks all keep their signals inside the normal operating range, the twin's residual moves by a median factor of 1.04 and the gate detects at most 7.07% of a quarantined class. The gate therefore detects attacks that place implausible values on the bus and not attacks that place plausible ones, and we report that boundary as measured rather than assumed.

INDEX TERMS Anomaly detection, automotive security, controller area network, digital twin, intrusion detection systems, open-set recognition, vehicular networks, zero-day attacks.

I. INTRODUCTION

The Controller Area Network (CAN), standardized as ISO 11898-1 [1], has been the foundation of in-vehicle communication for 30 years and was built for a threat model that is no longer valid. A CAN frame contains an arbitration identifier, a data length code, up to eight payload bytes, but no sender identity, no message authentication code and no encryption. Arbitration is based on the numeric value of the identifier only, so if an electronic control unit (ECU) sends a low identifier, it will always get the bus when it tries to use it. These properties are not bugs in any implementation, but rather properties of the protocol itself. The electric vehicle (EV) ups the ante. Braking, steering and propulsion in an EV are sent as digital commands on the same bus that carries diagnostics and telemetry, and drive-by-wire platforms make those commands available to any controller on the network. Koscher et al. [2] demonstrated that physical access to the bus allows arbitrary control of safety functions, Checkoway et al. [3] extended this to remote entry points, and Miller and Valasek [4] drove a Jeep Cherokee off a highway from a laptop. The solution has been to monitor the bus and raise an alarm if the traffic is not behaving as expected, as the protocol cannot be changed without replacing all the ECUs on the vehicle. An in-vehicle intrusion detection system (IDS) is what this is. There is a large body of work that uses machine learning (ML) and deep learning (DL) to solve that monitoring

problem, and the reported numbers are impressive: accuracies above 99% are commonplace [5]–[8]. Those numbers are also, in a particular way, uninformative. These are closed set results. The model is trained on a list of attacks, tested on the same list, and evaluated on the number of times it identifies the correct attack from the list. That protocol does not specify what to do if an attack comes along that wasn't in the catalogue, and that's what a deployed detector will eventually encounter. The gap is structural. A supervised classifier is constrained by its label set. It can only output one of k classes it has been trained on, and an attack that is not in the set is assigned to the class that fits it best. This is the open set recognition problem [9] and the complementary problem is anomaly detection based on a model of normal behaviour [10]. That is almost always the normal class for traffic that is not like a flood. This bound remains when there is additional convolutional or recurrent capacity, since the missing component is a reference of what the bus should be doing, without any catalogued attack. We create that reference as a digital twin. A gated recurrent unit (GRU) network is trained only on normal traffic and is used to predict the next payload for each arbitration identifier based on the history of that identifier. The residual is the difference between the prediction and the observation, and it is calculated for any frame, even if it is part of an attack that has not been labelled. A second decision channel is added to the supervised classifier, based on a threshold on that residual, calibrated using held-out normal frames only. The following are the contributions of this work. • Quarantine of an attack class at four stages: label space, each training context window, per-ECU history of the digital twin, and the calibration set of the anomaly threshold, for a leave-one-attack-out evaluation protocol for CAN intrusion detection. We show that removing the class from the labels alone leaves residual contamination, and we quantify how much. A two-view residual anomaly gate that normalises the digital-twin residual per arbitration identifier and per payload byte, with a frame-level view and an ECU-stream view. We demonstrate that it is this standardisation, not a global threshold, that makes a single bus-wide operating point usable. An empirical proof that the closed-set benchmark on GEM-CAN is saturated: the four architectures compared in this work are within 0.0031 macro-F1 of each other, while the same four are separated by more than 74 percentage points on a quarantined attack class. A defined scope of the method. The attack must increase the twin's residual above learnt-normal for detection, and, if it is held out, the steering tampering remains within the range the twin already accepts for its identifier. We report this limit with the same prominence as the positive result and quantify the condition under which the gate applies. The remainder of this paper is organised as follows. Section II summarizes the state of the art in machine learning, deep learning, unsupervised and specification-based intrusion detection for CAN, as well as the public datasets used to evaluate these techniques, and highlights their shortcomings. The GEM-CAN dataset and threat model are described in Section III. The proposed TwinShield-Fusion architecture is presented in Section IV, in the order of the processing stages. Section V describes the experimental setup, the leakage-controlled split and the leave-one-attack-out protocol.

II. RELATED WORK

There are two lines of research on in-vehicle intrusion detection. The first is the learning paradigm: supervised methods learn a decision boundary from labeled attack traffic, while unsupervised and specification-based methods learn normal behavior and detect deviations from it. The second is the feature level: frame-level methods read arbitration identifiers, payload bytes and inter-arrival times directly, while signal-level methods first decode payloads into physical signals. Rajapaksha et al. [11] review the literature and find that most of the published systems are supervised, frame-level and tested on a limited number of public datasets with closed-set conditions. We structure this section in the same way, naming the dataset used in each study and ending with a summary of what the body of work leaves unresolved..

A. CLASSICAL MACHINE LEARNING METHODS

Early learning-based detectors used traditional classifiers on handcrafted frame features.Early learning-based detectors used traditional classifiers on handcrafted frame features. On the HCRL Car-Hacking dataset, which includes denial-of-service (DoS), fuzzing and spoofing captures from a Kia Soul and a Chevrolet Spark, Bari et al. [12] achieved an accuracy of more than 99% for the tree-based classifiers. Lee et al. [13] adopted a protocol-level approach with OTIDS, which measures the offset ratio and time interval between a remote frame and the response it triggers, and the resulting OTIDS dataset, collected from a KIA Soul, has since been widely used as a benchmark. Müter and Asaj [14] did not even bother with learning and instead tracked the entropy of the message stream, which is able to detect volumetric injection on any vehicle without any training data. These methods are cheap and easy to interpret, and they do not transfer well to attacks that maintain the statistical profile of the bus. They are tested on captures with known attack classes, and the reported accuracy reflects recognition of known attacks, not detection of novel attacks.

B. DEEP LEARNING METHODS

Deep models eliminated the feature-engineering step. Kang and Kang [5] used a deep neural network for CAN payloads in a simulated OCTANE capture and achieved detection rates of more than 97%. To take advantage of the temporal

structure of injection, Song et al. [6] stacked 29 consecutive frames into a 2D grid and fed it into a reduced Inception-ResNet on the Car-Hacking dataset. On Car-Hacking, Seo et al. [7] introduced GIDS, which generates extra attack samples for training. Lokman et al. [8] summarize this family and point out that it is consistently dependent on labeled attack traffic. A newer strand is signal-level deep models. Shahriar et al. [15] introduced CANShield, a framework of convolutional autoencoders applied to the time-series of the decoded signals at multiple time scales, tested on the simulated SynCAN dataset and on the real-vehicle ROAD dataset, and designed to address masquerade attacks which are not covered by frame-level methods. Like our own gate, CANShield is not part of the supervised family, and its evaluation is based on attack-free traffic, not on an explicit quarantine.

C. UNSUPERVISED AND PREDICTION-BASED METHODS

A third line represents normal traffic and sets thresholds for the deviation from it. The closest antecedent to the mechanism we adopt is that of Taylor et al. [16] who train a long short term memory (LSTM) network to predict the next payload on each identifier and threshold the prediction error. Hanselmann et al. [17] take the concept further and propose an LSTM-and-autoencoder architecture over multiple decoded signals trained on attack-free traffic and tested on SynCAN and a real capture. Cho and Shin [18] go orthogonal and fingerprint ECUs based on clock skew, identifying the transmitter, not the content, which detects a masquerading ECU, but not a compromised genuine one. The evaluation is our contribution to [16] and [17] and not the mechanism. Both works report detection on attacks not included in training, and both report it for the unsupervised channel alone. Neither does it isolate how much of the detection is due to independence from the label by placing those that channel beside supervised baselines under an identical, auditable quarantine.

D. DIGITAL TWINS FOR SECURITY MONITORING

A digital twin is a model that is maintained in sync with a physical system [19]. Twins have been suggested for intrusion detection in industrial control environments where the physics of the process limits what sensor readings are possible. The principle remains the same: an ECU that sends a known message pattern can be modelled, and deviations from the model are evidence, although the constraints on CAN payloads on a road vehicle are less strict than in a chemical process. The difference between a twin and a generic autoencoder in this context is that the former is conditioned on the transmitting ECU, meaning that each stream is evaluated against its own learned behaviour.

E. PUBLIC DATASETS FOR CAN INTRUSION DETECTION

The public data sets for CAN intrusion detection are available.Public data sets for CAN intrusion detection are available. Advances in this area are limited by the number of captures. Car-Hacking [6] provides DoS, fuzzing and two classes of spoofing attacks from two passenger vehicles, and is the most popular benchmark; OTIDS [13] extends this with remote-frame and fuzzing attacks on a third. Signal-level ground truth is simulated in SynCAN [17] at the expense of realism. Verma et al. [20] presented ROAD, a 3.5-hour real traffic with 33 attacks, including masquerade scenarios, which were verified on the vehicle, and explained how the convenience of previous datasets has influenced and constrained the field. The first public capture from an autonomous electric vehicle, described in Section III, is used here and is called GEM-CAN [21, 22].

F. SUMMARY AND RESEARCH GAP

Table I summarises the studies discussed above by method, by the dataset on which each was evaluated, and by the assumption that bounds what its reported figures can establish. Three observations follow from it.

TABLE I
SUMMARY OF REPRESENTATIVE CAN INTRUSION DETECTION STUDIES

Study	Method	Dataset	Evaluation limitation
Kang and Kang [5]	Deep neural net	OCTANE (sim.)	Closed set
Song et al. [6]	Inception-ResNet	Car-Hacking	Closed set
Seo et al. [7]	GAN-augmented CNN	Car-Hacking	Closed set
Bari et al. [12]	SVM, tree, kNN	Car-Hacking	Closed set
Lee et al. [13]	Remote-frame offset	OTIDS	Known types only
Müter and Asaj [14]	Message entropy	Own capture	Volumetric only
Shahriar et al. [15]	Signal autoencoders	SynCAN, ROAD	No quarantine
Taylor et al. [16]	LSTM prediction error	Own capture	No baseline comparison
Hanselmann et al. [17]	CANet autoencoder	SynCAN, real	No baseline comparison
Cho and Shin [18]	Clock-skew fingerprint	Own capture	Masquerading ECU only
This work	Head + twin residual gate	GEM-CAN	Single vehicle capture

First, the accuracy is consistently high and consistently closed-set. The numbers in Table I are recognition rates for attacks in the attack catalogue and do not reflect the recognition of attacks not in the attack catalogue. Second, evaluation focuses on two or three public captures, most of which are dominated by volumetric injection, and the arbitration identifier is nearly enough to evaluate them; we quantify this shortcut in Section III-C. Third, the unsupervised entries are trained on attack-free traffic, but report their results separately from supervised baselines, leaving the comparison that matters for deployment unmade. These three observations form the gap that this paper aims to fill. We place a supervised classifier and an attack-agnostic residual gate on the same data, under one quarantine applied at four stages, and report both under a metric that is defined irrespective of label space.

III. DATASET AND THREAT MODEL

A. DATASET OVERVIEW

We test on GEM-CAN [21] [22] which is a public capture recorded by Tavasoli et al. from an autonomous electric vehicle (Polaris GEM e6) on a closed rural test track. The vehicle is equipped with a 250 kbit/s CAN bus and the traffic was recorded using a PCAN-USB interface connected to the vehicle via a PACMod drive-by-wire interface. Two captures are supplied: one of nominal autonomous driving, and one during which attacks were physically injected onto the live bus while the vehicle was stationary in autonomous mode, for safety. Every frame carries a timestamp, an arbitration identifier, a data length code, eight payload bytes and an attack-type label. This capture is appropriate for the question we ask because of three properties. It is not a simulation, so payload dynamics are real and not simulated. It is an electric, drive-by-wire platform, and the commands that are attacked are the commands that actually actuate the vehicle. And it contains both a volumetric attack and two low-rate tampering attacks on legitimate identifiers, which lets a single dataset exercise both the easy and the difficult detection regimes. The composition is shown in Table II. We do not claim to have any information on attacks other than the three classes in the dataset. A second dataset, SynCAN, is introduced in Section VI-E, where it is used to test whether these results generalise.

TABLE II
COMPOSITION OF THE GEM-CAN CAPTURE

Class	Frames	Arbitration IDs used
Normal	101,039	13 distinct IDs
DoS	41,080	00000000 only
Brake tampering	663	18FF00F9 (shared with normal)
Steering tampering	662	18FF70F9 (shared with normal)
Total	143,444	—

The two attack families are different in a way that determines all that follows. The denial-of-service (DoS) attack sends an arbitration identifier of 00000000 to the bus. Since CAN arbitration is based on the lowest identifier, this identifier is given priority over all other legitimate messages on the vehicle, and real traffic is starved, the bus load rises from 20% to 100% during the flood. It's also trivially separable: identifier 00000000 is not sent by any legitimate ECU on this vehicle, so the identifier alone is enough to identify the attack. Low rate of tampering with brakes and steering. They insert a few fake frames into the identifiers 18FF00F9 and 18FF70F9, which are used for real traffic during the capture, and the recorded effect is a steering-lock delay of about 20 seconds. The identifier is thus not informative as a discriminator, and detection must be based on the payload or on timing. The forged payload is well outside the normal range for brake tampering: bytes DATA_4 and DATA_5 range from about 0.5 to 58.8 and 77.5 respectively. The forged values are stored within the range of values that the identifier typically holds, a condition we will return to in Section VI-D for steering tampering.

B. THREAT MODEL

We assume an adversary who has already obtained write access to the CAN bus, whether through a physical diagnostic port, a compromised telematics unit or a subverted ECU, and who can transmit arbitrary frames with arbitrary identifiers. The opponent can jam the bus or create payloads for identifiers that are already used by legitimate ECUs. We assume that the detector is passive and cannot authenticate the senders, as there is no mechanism provided by the protocol for this. Any attack that does not modify the bus, e.g., passive eavesdropping, is not considered by any traffic-based detector and is not considered in this work..

C. ARBITRATION-IDENTIFIER BIAS ANALYSIS

Before training anything we audited the dataset for shortcuts, and found a severe one. A lookup table that guesses the majority class of each arbitration ID, based on no other information, and no learning whatsoever, achieves 98.9% accuracy on our test split. It has a per-class F1 of 1.00 on DoS and 0.00 on both tampering classes, giving a macro-F1 of 0.498. There are two things that follow, and they form the remainder of this paper. This is not a dataset that has a meaningful accuracy headline, so macro-F1 and per-class figures are reported throughout. The usefulness of any payload-modelling element should be evaluated based on the tampering classes, as DoS is already addressed with a lookup table. We also retrain all models without the arbitration ID in the input at all, which means that the shortcut can be removed instead of just recognized. The data set is also highly redundant. 87.4% of frames are byte-identical to at least one other frame, and 80.6% of test frames are exactly identical to a frame in the training split. This is inherent to CAN, where ECUs send out fixed status payloads at fixed intervals, but it makes any frame-level metric look worse and is another reason not to lead with accuracy.

IV. PROPOSED METHOD

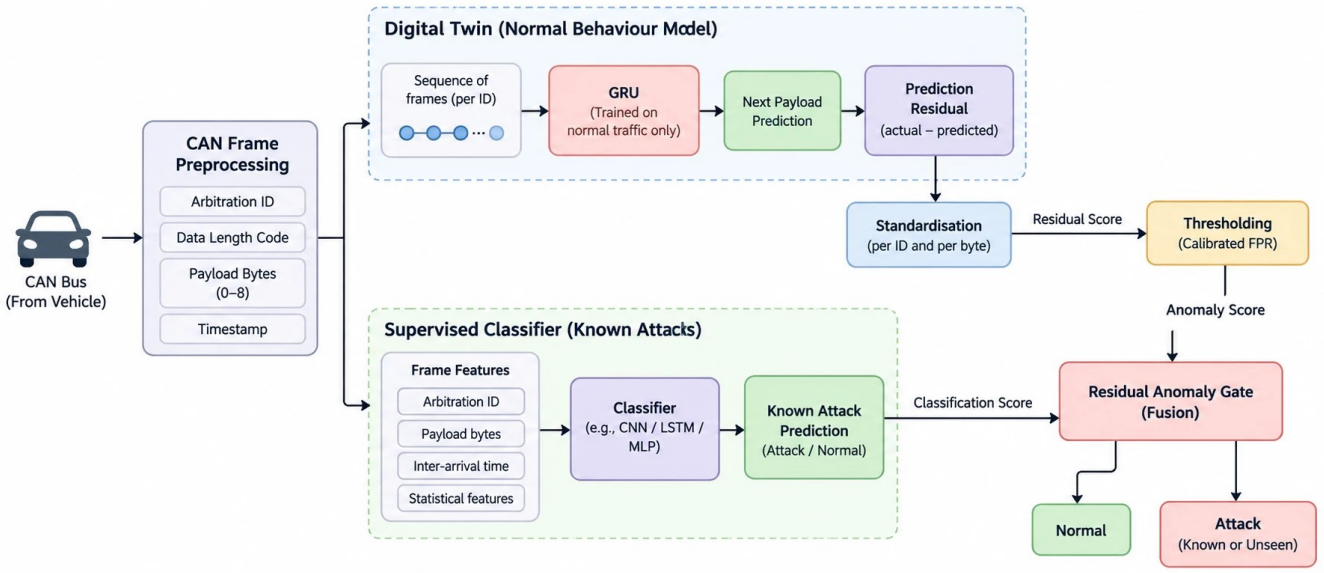


FIGURE 1. TwinShield-Fusion. Raw CAN traffic is preprocessed once and read by two branches. The digital twin predicts the next payload of each arbitration identifier from normal traffic alone, and its prediction residual is standardised per identifier and per byte and thresholded at a calibrated false-positive rate (Sections IV-C and IV-F). The supervised classifier, a CNN-BiLSTM in this work, names the attacks it was trained on (Section IV-D). The residual anomaly gate combines the two scores, which is what lets an attack with no label of its own still raise an alarm (Section IV-G).

A. SYSTEM OVERVIEW

Fig. 1 shows the architecture. Raw CAN traffic is split into two branches. The supervised branch constructs a 32-frame context window and classifies the current frame using a CNN-BiLSTM, which is enhanced by an attention-weighted fusion of per-ECU residual streams. The unsupervised branch feeds the digital-twin residual into an anomaly gate whose threshold was set with normal traffic. If either branch is burning, an alarm is raised by a frame. The supervised branch labels the attack if it recognises it; the gate can only say that something is wrong, which is the best it can do for an attack without a label.

B. CAN TRAFFIC AND CONTEXT WINDOW

Each frame is represented by its eight payload bytes scaled to [0,1], its data length code, the inter-arrival gap to the previous frame on the same identifier, and an index for the arbitration identifier itself. A single frame is not enough to distinguish a forged command from a legitimate one on the same identifier, so the supervised branch reads a context window of 32 frames before and after the frame under test, in bus order. The unit of classification is the window throughout, and the contents of the window also dictate what is to be excluded under the quarantine protocol of Section V-B.

C. GRU DIGITAL TWIN

Let $p_t \in [0,1]^8$ be the scaled payload of frame t and i_t its arbitration ID. The twin is a GRU [23] with 96 hidden units and a 16-dimensional embedding of the identifier, allowing a single network to be used for all ECUs. It predicts the next payload on identifier i_t, given the eight most recent payloads observed on this identifier. Training minimizes mean squared error to the true next payload, over normal frames from the training split only.

The residual is $r_t = | p_t - \hat{p}_t |$ *(1)*

Element-wise, that is, r_t is an 8-dimensional vector, one component for each payload byte. The twin achieves a mean absolute error of 0.041 in the units the model optimises, or 10.52 on the 0–255 byte scale, on held-out normal frames. That's not a trivial mistake, and it doesn't have to be. CAN payloads multiplex multiple independent signals, and some bytes are rolling counters or noisy sensor readings that are not predictable in principle. What detection needs is not a residual of zero, but a residual that is reliably different between normal and tampered traffic on the same identifier.

D. SUPERVISED CNN-BiLSTM CLASSIFIER

The supervised branch is a convolutional neural network (CNN) followed by a bidirectional long short-term memory (BiLSTM) network [24], a pairing chosen because the two capture complementary structure. Three one-dimensional convolutional layers read the context window and extract local byte patterns that are invariant to their position in the window, which is what a burst of injected frames looks like. The BiLSTM then reads the convolutional feature sequence in both directions, so a frame is judged against traffic that follows it as well as traffic that precedes it. A softmax head over the trained classes produces the class posterior.

This branch is the component that names an attack, and it is bounded by its label set in exactly the way described in Section I. We treat it as the recognition channel and make no

claim that it generalises beyond the classes it was fitted on; the experiments in Section VI-B measure precisely that bound.

E. ADAPTIVE MULTI-ECU FUSION

Each arbitration ID is treated as a separate ECU stream. Within a context window we summarise stream s by its mean residual, giving a tensor over streams, and mask out streams absent from the window. Additive attention [25] then weights the streams:

$$e_s = v^T \tanh(W h_s + b), \quad \alpha = \mathrm{softmax}(e) \tag{2}$$

and the fused vector Σ_s α_s h_s is concatenated with the CNN-BiLSTM embedding before the classification head. The intent is that evidence concentrates on whichever ECU is under attack rather than being diluted across the bus. We report in Section VI that on this dataset the attention contributes nothing measurable beyond the residual itself.

F. RESIDUAL ANOMALY GATE

The gate is the part that is not affected by known attacks. It reads the same residual through two views. The frame view asks how much a single frame differs from what its own ECU normally looks like, per byte:

$$z_{frame}(t) = \max_b \left[r_{t,b} - \mu_{i,b} \right] / \sigma_{i,b} \tag{3}$$

where i is the arbitration ID of the frame and b is the index of the eight payload bytes. This is the sharp channel: a forged payload moving one byte out of range is immediately visible and will not be averaged away by surrounding traffic. The ECU view asks what is the distance of a whole stream from the context window:

$$z_{ecu}(t) = \max_{s \in present} \left[\bar{v}_s(t) - \mu_s \right] / \sigma_s \tag{4}$$

and the gate is triggered if the larger of the two is above a threshold:

$$alarm(t) = 1 \left[\max(z_{frame}, z_{ecu}) > z^* \right] \tag{5}$$

The means and standard deviations are computed on normal frames of the training split. The threshold z* is the (1−α) quantile of the score over normal frames from the validation split, where α is the desired false-positive rate, which is 1% in all experiments reported here. No attack frame of any class takes part in either step. This is the whole point of the attack-agnostic description of the gate: there is no signature in it that could have been memorised, just a model of normal. If μ and σ were estimated on the twin's own training frames and the threshold was read off the same frames, the false-positive rate would be underestimated, since the twin is able to predict those frames better than any other frames. The target rate is meaningful when calibrated on held-out normal frames with the same μ and σ. Standardising per identifier and per byte is essential to the design. Some ECUs, and some bytes in an ECU's payload, have a high baseline residual, due to counters or noisy readings. They are continuously activated by a single global threshold. The reason for the bus-wide threshold is that each individual byte of each ECU is compared to its own normal behaviour.

G. ALARM DECISION RULE

Disjunction is the way the two branches are joined. An alarm is raised by a frame if the supervised head assigns it to any attack class, or if the gate score is greater than z*, or both. The outcome reported varies from channel to channel. If the supervised head fires, the system outputs the class it identified. If only the gate fires, the system outputs UNKNOWN ANOMALY along with the identifier and the payload byte that generated the largest standardised residual, the maximum that can be said about an attack for which there is no label. The disjunction is intentional and the cost is clear. The alarm rate of normal traffic in the union of two channels is at least as high as the alarm rate of either channel alone, and Section VI-C reports the increase in alarm rate, not the alarm rate itself.

V. EXPERIMENTAL SETUP

A. LEAKAGE-CONTROLLED SPLITTING

Consecutive CAN frames are almost identical, so a uniformly random split leaks almost-identical frames between training and test. Instead, we divide each capture into contiguous blocks of 256 frames and allocate whole blocks to training, validation and test, stratified by the rarest class in each block. The context window of a frame is only drawn within its own block, and no frame is visible to two splits. The resulting split contains 100,352 training frames, 21,504 validation frames, and 21,588 test frames, with test support of 15,063 normal, 6,288 DoS, 119 brake tampering and 118 steering tampering. It was thought about and rejected to split it purely chronologically. The tampering attacks are in the first 9% of the attack capture, leaving the test set empty of tampering frames and making the rarest classes unmeasurable if the attack is cut chronologically. Our scheme preserves the correlation of blocks that are adjacent in time, which we state as a limitation of the design and revisit in Section VII.

B. LEAVE-ONE-ATTACK-OUT PROTOCOL

One attack class is marked as unknown and is not included in any fitting stage. If the label is removed, there are three additional ways for the class to access a model. A frame labeled normal with 32-frame context window containing injected brake frames has still been able to show the detector what the attack looks like, and a normal frame with per-ECU history containing the injected brake frames has still been

able to show the digital twin. We therefore use four different exclusions: 1) The detector head is k−1 way, meaning that there is no output index at all; 2) all training and validation frames whose context window contains a frame of the class are dropped; 3) the digital twin is refitted from scratch on normal frames whose per-ECU history is free of the class; and 4) the gate's μ, σ and threshold are taken from normal frames only, as described in Section IV-F. The test split is left untouched and still contains all 119 held-out brake tampering frames. Since one model has a label and the others do not, accuracy and F1 cannot be compared between models. Instead, we ask the question that any deployed detector must answer, which is well defined in any label space: did you raise an alarm? The alarm rate is reported for each true class, that is, the detection rate for attack classes and false-positive rate for normal class.

C. TRAINING CONFIGURATION

All detectors use a 32-frame context window, a batch size of 512, the Adam optimiser at a learning rate of 10^{-3}, and class-weighted cross-entropy to compensate for the 152:1 imbalance between normal and the tampering classes. The digital twin is trained for 5 epochs and the detectors for 6 epochs, and the weights are chosen based on the best validation macro-F1, and then tested once on the test split. All experiments run on CPU with a fixed seed of 42.

VI. RESULTS

A. CLOSED-SET CLASSIFICATION RESULTS

Table III shows the macro-F1 for each model on the common held-out test set, both with and without the arbitration ID as an input. The four architectures with the ID included span 0.0031 macro-F1 in the leave-one-attack-out experiment, and all five rows of Table III span 0.0041. Neither of these numbers is proof that one architecture is superior to another, it's proof that the benchmark has ceased to measure architecture. A lookup table based on the identifier alone achieves 98.9% accuracy, so there is little label information left for a sequence model to add.

TABLE III
CLOSED-SET MACRO-F1 ON THE SHARED HELD-OUT TEST SPLIT

Model	CAN ID included	CAN ID excluded
CNN	0.9968	0.9914
BiLSTM	0.9948	0.9895
CNN-BiLSTM	0.9979	0.9914
GRU twin + CNN-BiLSTM	0.9989	0.9979
TwinShield-Fusion	0.9979	0.9979

It is more informative to remove the arbitration ID. There the digital-twin residual boosts the CNN-BiLSTM backbone from 0.9914 to 0.9979 macro-F1. That gain should be measured against the support it is based on: macro-F1 is weighted heavily by the two tampering classes, which contain 119 and 118 test frames, respectively, so a single frame changing class moves a tampering F1 by approximately 0.008 and macro-F1 by approximately 0.002. A difference of 0.0065 is thus of the order of three frames. We report it as a consistent direction, not as a precisely resolved quantity, and Section VII tells what would be required to resolve it. The ablation in Table IV is not monotonic, and we read it as it stands. The macro-F1 drops from 0.9979 to 0.9958 when the scalar twin output is added, and only the full per-byte residual recovers and surpasses the backbone at 0.9989; if the identifier is not included the ladder does rise at each step. In both variants (0.9979 → 0.9979), adaptive fusion adds nothing to the residual. In this experiment, the gain is not in the attention layer, but in the residual, and we report it that way.

TABLE IV
ABLATION ON THE SAME CNN-BILSTM BACKBONE (MACRO-F1)

Configuration	CAN ID incl.	CAN ID excl.
CNN-BiLSTM	0.9979	0.9914
+ digital twin (scalar)	0.9958	0.9965
+ residual vector	0.9989	0.9979
+ adaptive fusion	0.9979	0.9979

B. UNKNOWN-ATTACK DETECTION

The main result is presented in Table V and Fig. 2. The three supervised baselines correctly identify 0.84%, 0.00% and 25.21% of the 119 frames that were not used in training. TwinShield-Fusion flags all 119.

TABLE V
LEAVE-ONE-ATTACK-OUT: BRAKE TAMPERING QUARANTINED FROM EVERY TRAINING STAGE

Model	Detected	False alarms	Known macro-F1
CNN	0.84%	0.03%	0.6667
BiLSTM	0.00%	0.08%	0.7607
CNN-BiLSTM	25.21%	0.31%	0.7787
TwinShield-Fusion	100.00%	2.26%	0.7829

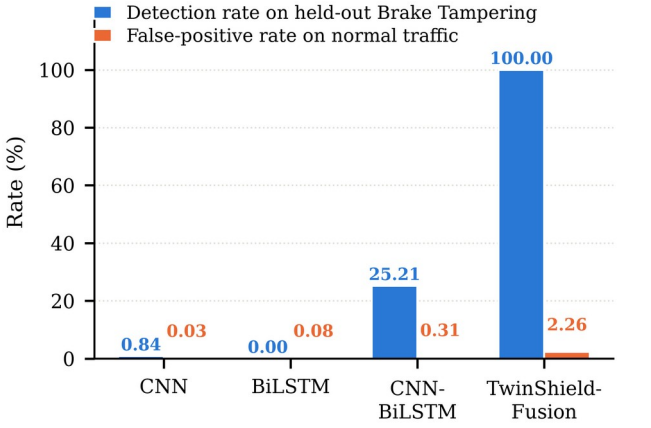


FIGURE 2. Detection of a quarantined attack class. Brake tampering was removed from the label space, from every training context window, from the digital twin's per-ECU history and from the gate's calibration set. Detection rate is measured on its 119 held-out test frames; the false-positive rate is measured on 15,063 normal test frames.

The credit is for a particular component and the decomposition is worth stating. TwinShield-Fusion's

supervised head contributes none of that detection, it has no label to assign, and the anomaly gate contributes all of it. The generaliser is the residual, the nameer is the classifier. It would be a mistake to give the credit to the architecture. The 25.21% of CNN-BiLSTM is worth commenting on, as the mechanism is not recognition but misattribution. That model also doesn't have an output for brake tampering. What it does is use some of those frames for steering tampering, the other tampering class it was trained on, which happens to have an identifier family and a timing signature. The alarm is set by mistake. The two weaker baselines can't even do that.

C. FALSE-ALARM COST OF THE RESIDUAL GATE

The cost of the second channel is the false-positive cost. The always-on alarm detects all attacks, so a detection rate without false positive is not a result. On normal test traffic, ours is 2.26%, while the baselines range from 0.03% to 0.31%, or about 340 false alarms per 15,063 normal frames. This is the cost of the additional channel and it is not insignificant. The target rate at calibration is 1%, but the realised rate is higher due to the fact that the false alarms of the supervised head are also counted as false alarms of the gate, and due to the fact that the normal traffic used for validation is not the same as the normal traffic used for test. 2.26% is 22.6 false alarms per thousand normal frames. The unit that is operationally meaningful is alarms per unit time, which on a bus with f frames per second is 0.0226f. At any frame rate a production powertrain bus maintains, there are more alarms than an operator or a downstream controller could triage, so we present the gate at this operating point as a research instrument, not as a deployable alarm. The threshold is the knob available, and the trade-off a deployment must make is the one that the curve represents, as it moves down the curve by lowering the target false-positive rate. We have not determined what the tolerable alarm rate is on it, as it is dependent on a per-vehicle frame rate and a per-fleet triage budget that this capture does not provide. The two experiments are compared in Fig. 3 and the argument of this paper is summarised. The four models are not distinguishable on the closed set. On the quarantined class they are more than 74 percentage points apart. A closed set score is a score that only considers attacks that have already been classified.

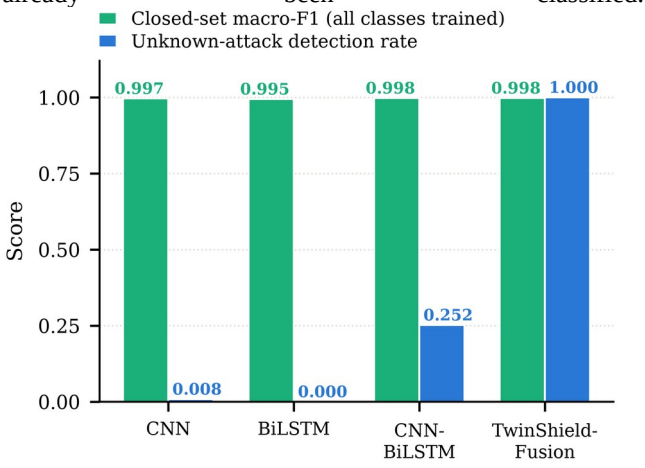


FIGURE 3. The same four models under two protocols. Closed-set macro-F1 separates them by 0.0031; detection of a quarantined attack separates them by more than 74 percentage points.

D. CONDITIONS UNDER WHICH THE RESIDUAL SEPARATES

The operating condition of the residual signal is worth showing. The gate identifies an attack if that attack sends payload bytes that are not what the twin has learned is normal for the identifier. This is a real condition, not a formality, and Fig. 4 illustrates the case where it is satisfied and where it is not.

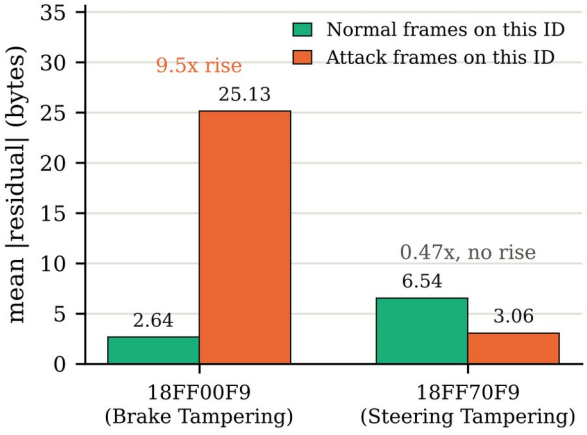


FIGURE 4. Mean digital-twin residual on the two tampering identifiers. Brake tampering raises the residual by a factor of 9.5 and is detected. Steering tampering leaves it below the normal level and is not, because the forged payloads fall inside the range the twin already accepts for that identifier.

Under brake tampering, the mean residual is 25.13 bytes on identifier 18FF00F9, which is 9.5 times the 2.64 bytes measured on normal frames of that identifier. The twin localises the attack exactly: bytes DATA_4 and DATA_5 contain nearly all of the increase. On 18FF70F9 the mean residual under steering tampering is 3.06 bytes, below the 6.54 bytes of normal traffic on the same identifier. The forged steering payloads are within the range that the twin already accepts for that identifier. The gate is therefore unable to detect steering tampering, and this is not an

experiment, but a measurement: a score that is above learnt-normal is a positive deviation, and a positive deviation is what the gate is looking for. This restriction on the method is worth stating exactly. Detection does not depend on knowing what the attack is, but it does depend on the attack being anomalous in the payload. These are different conditions, and mixing them up would overstate the method.

E. CROSS-DATASET EVALUATION ON SYNCAN

A result on one capture is a result on one capture. We therefore repeated the whole protocol on a second, independent dataset: SynCAN [17], the synthetic CAN benchmark released with CANet. SynCAN is signal-space rather than payload-space, in that each frame carries one to four already-normalised signal values instead of eight raw bytes. We pack each signal into the payload the way a vehicle database packs one, as a 16-bit big-endian integer occupying bytes 2k and 2k+1. At most four signals per identifier fits the eight-byte payload exactly, the quantisation step is 1/65535, and the input format of every model is unchanged. Three of SynCAN's attack types take the three attack slots of our label space: flooding, a high-rate injection on an existing identifier, stands in for DoS; plateau, a signal held at a constant value, and continuous, a signal drifting slowly away from its true value, stand in for the two tampering classes. Playback and suppress have no slot and are excluded. Every number in this subsection describes a SynCAN attack: the slot names are ours and the attacks are not GEM-CAN's. The converted capture holds 250,000 frames, 223,222 of them normal, and is split by the block-wise procedure of Section V-A.

The identifier shortcut does not exist on this dataset. The audit of Section III-C, repeated here, gives 89.29% for a lookup table on the arbitration identifier alone, which is exactly the proportion of normal frames in the capture: the table has degenerated into always answering Normal. Every SynCAN attack rides on an identifier that already carries legitimate traffic, so the identifier carries no attack information at all. Table VI shows the consequence. Removing the identifier improves three of the five models instead of harming them, and changes the proposed system by 0.0020; only BiLSTM, the weakest architecture on either dataset, loses materially, by 0.1092. On GEM-CAN the same pair of columns is the difference between a benchmark that measures architecture and one that does not. The twin reaches a mean absolute error of 9.73 bytes on 33,516 held-out normal SynCAN frames, against 10.52 on GEM-CAN, so the predictor itself transfers.

TABLE VI
CLOSED-SET MACRO-F1 ON SYNCAN

Model	CAN ID included	CAN ID excluded
CNN	0.9012	0.8771
BiLSTM	0.8456	0.7364
CNN-BiLSTM	0.8842	0.9028
GRU Digital Twin + CNN-BiLSTM	0.8452	0.9183
TwinShield-Fusion	**0.8885**	**0.8905**

The residual gate does not. Table VII repeats the leave-one-attack-out protocol once for each of the three attack slots. On GEM-CAN the gate raises an alarm on 100.00% of the quarantined brake-tampering frames. On SynCAN it raises an alarm on 7.07% of quarantined flooding frames, 1.56% of quarantined plateau frames and 2.60% of quarantined continuous frames, at false-positive rates of 2.94%, 1.47% and 1.45% respectively. The three supervised baselines remain worse, between 0.00% and 2.44%, so the ordering of the four systems survives the change of dataset. The absolute claim does not. Known-class macro-F1 stays between 0.8347 and 0.9698 across the twelve runs, so the reduced label space is learned on SynCAN as well as it is on GEM-CAN, and the gap is in the gate alone.

TABLE VII
LEAVE-ONE-ATTACK-OUT ON SYNCAN. EACH ATTACK SLOT IS QUARANTINED FROM EVERY TRAINING STAGE IN TURN; EACH CELL GIVES THE DETECTION RATE ON THE QUARANTINED CLASS AND THE FALSE-ALARM RATE ON NORMAL TRAFFIC, IN PERCENT

Model	flooding	plateau	continuous
CNN	0.35 / 1.16	0.00 / 1.39	0.65 / 0.48
BiLSTM	1.72 / 2.76	2.44 / 2.92	0.91 / 1.36
CNN-BiLSTM	0.16 / 1.94	0.59 / 1.71	0.78 / 0.40
TwinShield-Fusion	**7.07 / 2.94**	**1.56 / 1.47**	**2.60 / 1.45**

The reason is visible in the residual itself, and it is the boundary this paper has already named. On GEM-CAN, brake tampering writes payload bytes outside the range the twin has ever observed for that identifier, and the mean residual rises from 2.64 to 25.13 bytes, a factor of 9.5. No SynCAN attack does that. A plateau holds a signal at a constant, valid value, and a next-value predictor finds a constant more predictable than a moving signal, not less. A continuous drift is smooth and therefore also predictable one step at a time. Flooding repeats an existing identifier at high rate with values that are themselves legitimate, so its signature lies in the arrival rate, which a payload residual does not observe. Across the thirty identifier-and-class pairs of Fig. 5 the ratio of mean attack residual to mean normal residual runs from 0.54 to 2.62 with a median of 1.04: the twin is no more surprised by an attacked frame than by a normal one. A gate calibrated to a 1% false-alarm rate on normal traffic cannot separate distributions that overlap that closely, and no threshold on this residual would.

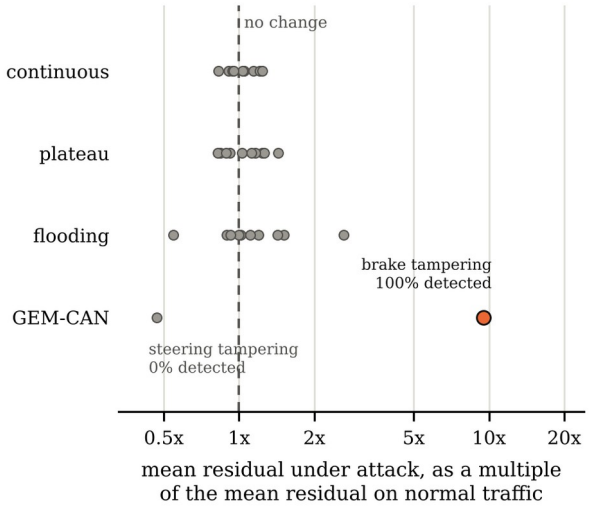


FIGURE 5. Why the gate transfers or does not. Each marker is one arbitration identifier under one attack class, placed at the ratio of its mean digital-twin residual under attack to its mean residual on normal traffic. Only brake tampering on GEM-CAN moves the residual far enough for a gate calibrated at a 1% false-alarm rate to separate it; all thirty SynCAN identifier-and-class pairs lie between 0.54x and 2.62x.

The quantisation is not the cause. Repeating the plateau run with each signal packed into a single byte instead of two lowers the twin's mean absolute error from 9.20 to 1.70 bytes, because the low byte of the 16-bit encoding is largely unpredictable noise; detection of the quarantined class is nevertheless identical at 1.56%, and the median residual ratio moves only from 1.12 to 1.08. A five-fold more accurate predictor buys nothing here, because the attacked frames are not the frames it predicts badly. The outcome is the failure mode of Section VI-D, where steering tampering on GEM-CAN lowered the residual instead of raising it; SynCAN consists entirely of attacks of that kind. The honest statement of scope is therefore narrower than the GEM-CAN result alone would support. The digital-twin residual gate detects attacks that put implausible values on the bus and does not detect attacks that put plausible values on it. Which of the two a deployment faces is a property of the adversary rather than of the detector, and a system that must handle both has to pair this gate with a timing or sequence model. We report this as a measured limit of the method, not as a property of SynCAN.

F. COMPUTATIONAL COST

A forward pass through all four models and one gate evaluation requires 12.0ms per frame on CPU, which is about 3ms per model. The gate itself is two standardisations and a comparison, and the cost is minimal compared to the twin's forward pass. The system has not been designed for embedded use and we do not guarantee line-rate operation on vehicle hardware.

VII. DISCUSSION AND LIMITATIONS

We report generalisation across attack classes within a single capture, and Section VI-E extends that to a second dataset. It is still not generalisation across vehicles or drive cycles, which are different claims again. GEM-CAN is one vehicle on one closed track, and our figures are within-capture performance. A second boundary needs to be stated. Our models do not know about brake tampering, but they do know about its arbitration identifier. The twin has a normal profile for 18FF00F9, and the residual is measured against that profile. If an attack comes in on an identifier that the twin has never seen, it will be treated as the unknown-ID embedding, which we have not tested. A replay attack would probably be able to avoid the gate for the same reason as steering tampering. If a true payload is replayed, the twin receives a predictable payload, and the residual remains low. Replay detection is not payload modelling, but rather timing or sequence modelling, which is better suited to the supervised branch. Interpretability is partial. The gate reports the identifier and payload byte carrying the largest standardised residual, which localises an alarm to an ECU and a signal, and we compute Shapley additive explanation values [26] over the supervised branch's inputs to attribute its decisions. Neither device explains why the twin finds a given payload surprising, and a full account of that would require decoding the payload into physical signals, which the vehicle's database would be needed to do. Lastly, the block-wise split results in temporally correlated adjacent blocks, and 87.4% of the frames in the dataset are byte identical to another frame. Both overestimate frame-level metrics. Both have an impact on the leave-one-attack-out comparison, which is the same for all four models, but both should temper any reading of the absolute closed-set figures. Four limits of the evaluation itself should be stated along with those of the method, as each limit places a limit on how far the headline comparison can be pushed. One quarantined class is used in the experiment. The class of brake tampering was held out because it is the class whose residual behaviour the twin can represent; steering tampering, held out in its place, is the negative case reported in Section VI-D. This dataset would support a full leave-one-attack-out sweep over each class, but this would be a characterisation of the method rather than an illustration. All figures are from a single seeded run. The difference carrying the argument, 74 percentage points on the quarantined class, is much greater than any reasonable seed variance; the differences in Tables III and V, of the order of a few frames, are not. Those tables should include confidence intervals for repeated seeds before any ordering among the four architectures is claimed, and we claim none. The baseline set is three supervised architectures. It does not contain the unsupervised comparisons that a reader might reasonably expect to make: a plain autoencoder, a one-class model, an explicit open-set scoring rule, or a per-identifier range check, which is the simplest thing that could detect brake tampering on this capture. The only unsupervised number we report is the twin residual, because the decomposition in Section VI-B assigns all of the quarantined-class detection to the gate and none to the supervised head. Validation covers two datasets. GEM-CAN carries the positive result and SynCAN, in Section VI-E, carries the negative one. Car-Hacking, OTIDS and ROAD

remain untested, and each would need the same conversion and the same four-stage quarantine before it could be reported. We do not extrapolate beyond the two we ran.

VIII. CONCLUSION

Closed-set evaluation has announced to the CAN intrusion detection literature that the problem is almost solved. The four architectures compared here are within 0.0031 macro-F1 of each other on the dataset studied here, and this is more a reflection of the benchmark than the models. The same four models are separated by over 74 percentage points when trained in a leave-one-attack-out protocol that quarantines an attack class from the label space, the digital twin's history and the anomaly threshold's calibration set. The digital-twin residual gate, which is calibrated on normal traffic only, achieves a 2.26% false-positive rate for every held-out frame of the quarantined class, whereas the best supervised baseline achieves a 25.21% false-positive rate. The mechanism has a well-defined boundary and we have measured it, not asserted it: detection is based on the fact that the attack is anomalous in the payload, and an attack whose forged values fall within the learnt-normal range is not detected. Section VI-E measures that boundary on a second dataset, SynCAN, whose attacks are all of that kind and of which the gate detects at most 7.07%. Future work should confirm across vehicles and drive cycles, and improve the residual representation for attacks that mimic normal payload statistics, where the remaining headroom is.

REFERENCES

[1] Road Vehicles—Controller Area Network (CAN)—Part 1: Data Link Layer and Physical Signalling, ISO Standard 11898-1, 2015.
[2] K. Koscher et al., "Experimental security analysis of a modern automobile," in Proc. IEEE Symp. Security Privacy, Oakland, CA, USA, 2010, pp. 447–462.
[3] S. Checkoway et al., "Comprehensive experimental analyses of automotive attack surfaces," in Proc. 20th USENIX Security Symp., San Francisco, CA, USA, 2011.
[4] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," Black Hat USA, Las Vegas, NV, USA, Tech. Rep., 2015.
[5] M.-J. Kang and J.-W. Kang, "Intrusion detection system using deep neural network for in-vehicle network security," PLoS ONE, vol. 11, no. 6, Jun. 2016, Art. no. e0155781.
[6] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," Veh. Commun., vol. 21, Jan. 2020, Art. no. 100198.
[7] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in Proc. 16th Annu. Conf. Privacy, Security Trust (PST), Belfast, U.K., 2018, pp. 1–6.
[8] S. F. Lokman, A. T. Othman, and M. H. Abu-Bakar, "Intrusion detection system for automotive Controller Area Network (CAN) bus system: A review," EURASIP J. Wireless Commun. Netw., vol. 2019, Dec. 2019, Art. no. 184.
[9] C. Geng, S.-J. Huang, and S. Chen, "Recent advances in open set recognition: A survey," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 10, pp. 3614–3631, Oct. 2021.
[10] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Comput. Surv., vol. 41, no. 3, Jul. 2009, Art. no. 15.
[11] S. Rajapaksha, H. Kalutarage, M. O. Al-Kadri, A. Petrovski, G. Madzudzo, and M. Cheah, "AI-based intrusion detection systems for in-vehicle networks: A survey," ACM Comput. Surv., vol. 55, no. 11, Feb. 2023, Art. no. 237.
[12] B. S. Bari, K. Yelamarthi, and S. Ghafoor, "Intrusion detection in vehicle controller area network (CAN) bus using machine learning: A comparative performance study," Sensors, vol. 23, no. 7, Mar. 2023, Art. no. 3610.
[13] H. Lee, S. H. Jeong, and H. K. Kim, "OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame," in Proc. 15th Annu. Conf. Privacy, Security Trust (PST), Calgary, AB, Canada, 2017, pp. 57–5709.
[14] M. Müter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in Proc. IEEE Intell. Vehicles Symp. (IV), Baden-Baden, Germany, 2011, pp. 1110–1115.
[15] M. H. Shahriar, Y. Xiao, P. Moriano, W. Lou, and Y. T. Hou, "CANShield: Deep-learning-based intrusion detection framework for controller area networks at the signal level," IEEE Internet Things J., vol. 10, no. 24, pp. 22111–22127, Dec. 2023.
[16] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in Proc. IEEE Int. Conf. Data Sci. Adv. Anal. (DSAA), Montreal, QC, Canada, 2016, pp. 130–139.
[17] M. Hanselmann, T. Strauss, K. Dormann, and H. Ulmer, "CANet: An unsupervised intrusion detection system for high dimensional CAN bus data," IEEE Access, vol. 8, pp. 58194–58205, 2020.
[18] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in Proc. 25th USENIX Security Symp., Austin, TX, USA, 2016, pp. 911–927.
[19] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405–2415, Apr. 2019.
[20] M. E. Verma et al., "A comprehensive guide to CAN IDS data and introduction of the ROAD dataset," PLoS ONE, vol. 19, no. 1, Jan. 2024, Art. no. e0296879.
[21] M. Tavasoli, A. Sarrafzadeh, A. Karimoddini, T. Phuapaiboon, M. Khaleghi, and D. Tobias, "GEM-CAN: A real-world dataset of CAN-bus attack scenarios on an autonomous vehicle for intrusion-detection research," Data Brief, vol. 66, Jun. 2026, Art. no. 112805.
[22] M. Tavasoli et al., GEM-CAN: Real-World CAN-Bus Attack Scenarios on an Autonomous Vehicle for Intrusion Detection (Version 1) [Data set], Zenodo, 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17834776
[23] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), Doha, Qatar, 2014, pp. 1724–1734.
[24] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
[25] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in Proc. Int. Conf. Learn. Represent. (ICLR), San Diego, CA, USA, 2015.
[26] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, USA, 2017, pp. 4765–4774.

FIRST A. AUTHOR received the B.E. degree in computer science and engineering from [Institution], [City], [Country], in [year]. [One or two sentences on research interests, relevant project work and any awards. Replace this placeholder before submission.]

SECOND B. AUTHOR received the B.E. degree in computer science and engineering from [Institution], [City], [Country], in [year]. [One or two

sentences on research interests, relevant project work and any awards. Replace this placeholder before submission.]

THIRD C. AUTHOR received the B.E. degree in computer science and engineering from [Institution], [City], [Country], in [year]. [One or two sentences on research interests, relevant project work and any awards. Replace this placeholder before submission.]